



Rapport - Projet de Traduction Des Langages

Année: 2024-2025

Membres de l'équipe:

RAGOT Cyrian

BARILLY Victor

ENSEEIHT
Département SN
Janvier 2025

Contents

1	Introduction	2
2	Généralités	3
3	Pointeurs	3
3.1	Description	3
3.2	Implémentation	4
3.3	Tests	6
3.3.1	Passe gestion des identifiants	6
3.3.2	Passe typage	7
3.3.3	Passe placement mémoire	8
3.3.4	Passe génération de code	8
4	Variables globales	9
4.1	Description	9
4.2	Implémentation	9
4.3	Tests	9
4.3.1	Passe gestion des identifiants	9
4.3.2	Passe typage	9
4.3.3	Passe placement mémoire	9
4.3.4	Passe génération de code	9
5	Variables statiques locales	9
6	Paramètres par défaut	9
7	Conclusion	9
7.1	Difficultés rencontrées	9
7.2	Pour aller plus loin	10

1 Introduction

Ce projet a été mené dans le cadre des cours de Traduction Des Langages et de Programation Fonctionnelle. Il consiste au développement d'un langage de programmation nommé RAT qui fonctionne sur le langage assembleur simplifié TAM et au développement du compilateur associé permettant de traduire un programme RAT en instructions TAM exécutables. De plus, il est entièrement développé en Caml, langage fonctionnel adapté à l'écriture d'un compilateur.

Le lexique du langage RAT est définie par un lexer (lexer.mll) et sa grammaire par un parser (parser.mly) en utilisant Menhir, c'est la partie de définition du langage RAT.

La sémantique (la partie compilateur du projet) est développée en plusieurs passes d'analyse du code RAT :

- passe de gestion des identifiants : vérifier la bonne utilisation des identifiants de variables ou de fonctions lors de déclarations/affectations/appel
- passe de typage : vérifier que le programmeur a correctement typé ses variables, paramètres de fonctions, retours et qu'il utilise des opérateurs dont les types sont corrects
- passe de placement mémoire : déterminer les adresses des variables stockées en mémoire qui dépendent de la taille de chaque type de variable
- passe de génération de code : traduction de chaque instruction en code TAM exécutable

Pour développer ce compilateur, nous utilisons une structure de données appelée Arbre Syntaxique Abstrait (AST) qui contient les informations importantes à la compilation pour chaque passe. Entre autre, il ne contient pas les informations inutiles de l'arbre de dérivation qui est plus dur à utiliser.

Nous nous concentrons ici sur l'ajout de 4 fonctionnalités à notre langage et notre compilateur : les pointeurs, les variables globales, les variables statiques locales et les paramètres par défaut.

Pour chacun de ces ajouts, nous expliquerons leur concept et implémentation pour chaque passe du compilateur et les tests effectués sur chaque passe pour valider notre programme.

2 Généralités

Les tests d'intégration pour chaque ajout d'une fonctionnalité et pour chaque passe peuvent être trouvés dans les dossiers **tests/(nom_passe)/avec_(nom_ajout)** et l'on pourra lancer les tests avec la commande :

dune runtest tests/(nom_passe)/avec_(nom_ajout)

ou pour tous les tests avec la commande :

dune runtest

On donne aussi dans le rapport le nom des tests et leur description courte ainsi qu'une indication (OK) si le test correspond à un programme qui ne lève pas d'erreur et une indication (KO) dans le cas contraire.

3 Pointeurs

3.1 Description

On introduit les pointeurs à la manière de C.

Le référencement d'une variable sera nommé *Adresse* et se codera *&id*, c'est une nouvelle expression de la grammaire.

On ajoute aussi un nouveau non terminal à la grammaire : les affectables noté *A* et la règle de production comme définie dans le sujet. On code un déréférencement d'une variable comme suit : $(*A)$ et on la nommera *Dereference* dans le compilateur.

Il faut alors aussi modifier la règle de production pour l'instruction d'affectation et les identifiants dans les expressions deviennent un affectable.

Les pointeurs ont un type spécial nommé *Pointeur* sur un type, se type pointeur s'écrit avec une étoile sur un type. Il peut donc y avoir des pointeurs de pointeurs de manière récursive. Un tel type se codera *int** en RAT.

Un pointeur null sera nommé *Null* et se codera *null* en RAT.

Un nouveau pointeur pourra être déclaré par référencement mais aussi par allocation d'une nouvelle case mémoire pour une variable non initialisée. On écrira *New* cette opération et on la codera (*newint*) en RAT pour un entier par exemple.

Les pointeurs définis ainsi peuvent avoir un aspect récursif dans leurs appels, déclarations et affectations.

On donne les nouveaux jugements de typage pour les pointeurs ci-dessous :

$$\sigma \vdash null : \text{Pointeur}(\text{Undefined})$$

$$\begin{array}{c}
\frac{\sigma \vdash TYPE : \tau}{\sigma \vdash TYPE* : Pointeur(\tau)} \\
\\
\frac{\sigma \vdash TYPE : \tau}{\sigma \vdash (new \ TYPE) : Pointeur(\tau)} \\
\\
\frac{\sigma \vdash a : Pointeur(\tau)}{\sigma \vdash *a : \tau} \\
\\
\frac{\sigma \vdash a : \tau}{\sigma \vdash \&a : Pointeur(\tau)} \\
\\
\frac{\sigma \vdash E1 : Pointeur(\tau1) \quad \sigma \vdash E2 : Pointeur(\tau2)}{\sigma \vdash (E1 = E2) : bool}
\end{array}$$

Et les jugements de typages déjà existant qui ont été modifiés :

$$\frac{\sigma \vdash a : \tau \quad \sigma \vdash E : \tau}{\sigma, \tau_r \vdash a = E : void, []}$$

3.2 Implémentation

Lexer

On a les nouveaux terminaux REF, NULL et NEW.

Parser

On a de nouvelles règles de productions, les affectables, le nouveau type pointeur, les nouvelles expressions. (voir parser.mly)

Ast

De manière générale, nous avons dû modifier l'ast au niveau de l'instruction d'affectation pour toutes les passes, ajouter le type affectable qui peut être soit un *Ident* soit une *Dereference* et ajouter les expressions *New*, *Null*, *Adresse* et *Affectable*.

Nous avons aussi modifié le module utilitaire PrinterAstSyntax pour afficher les nouveaux symboles introduits par les pointeurs.

Type

Pour implémenter les pointeurs il a fallu modifier le module Type pour ajouter le type *Pointer of typ* et modifier les fonctions *getTaille*, *est_compatible_list* et *est_compatible*. Nous avons ajouté des tests unitaires pour tester ces fonctions avec les pointeurs. (voir fichier type.ml)

On remarquera surtout que les comparaisons entre type de pointeurs utilisent le type *Undefined*, ceci sert à la passe de typage pour vérifier que le type de *null* est compatible avec les autres types de pointeurs (par exemple un type *Pointer(Int)* est compatible à *Pointer(null)* lors de l'affectation $int * x = null$).

Fonctionnalités

Ensuite, un pointeur null a été implémenté comme un entier valant 0 et typé comme un pointeur sur *Undefined* (voir passes typage et génération de code). Ainsi, nous pouvons comparer des pointeurs à un pointeur null (voir plus loin). Nous avons aussi pensé à utiliser la primitive "MVoid" de TAM pour exprimer une adresse non initialisée mais cela rendait la comparaison de pointeurs compliqué, surtout que nous ne savions pas exactement comment se comporte cette primitive.

Nous avons choisis d'implémenter l'affichage des pointeurs qui affiche l'adresse stockée par celui-ci. Il existe donc un nouveau type d'instruction *AffichagePointer* dans *AstType* et *AstPlacement* ainsi que des nouveaux traitements d'instruction dans les passes de placement et de génération de code.

Aussi, nous avons implémenté l'opération binaire d'égalité entre pointeurs qui compare l'entier représentant l'adresse d'une variable contenu dans le pointeur. Cela donne aussi une utilité à la valeur null, on peut savoir si on pointeur ne pointe pas encore sur une variable de cette manière. Pour cela, il a fallu ajouter l'opérateur binaire *EquPointer* dans l'*AstType* et nous avons ajouté le traitement de cette opération binaire dans les passes de typage et de génération de code.

Nous avons pensé à ajouter une exception lorsque l'utilisateur tente de déréférencer une variable qui n'est pas un pointeur, on a nommé cette exception *TypeNonPointeur* et on peut retrouver la levée d'une telle exception dans la passe de typage dans la fonction *analyser_type_affectable*. Certains tests sur la passe de typage prennent en compte cette exception (voir les tests sur les pointeurs plus bas)

Les affectables dans les passes

On peut noter que les passes de gestion des identifiants, typage et génération de code possèdent maintenant une fonction pour traiter les affectables.

Le paramètre *modif* indique si l'affectable se trouve à droite ou à gauche d'une affectation, elle est utile pour vérifier qu'on ne tente pas de modifier une constante.

Aussi, le paramètre *isDeref* dans la passe de génération de code est utilisé pour

ne pas boucler plusieurs fois lors d'un déréférencement récursif de pointeurs de pointeurs. Mais ceci nous mène à parler du problème suivant.

Récursivité sur les pointeurs

Comme la grammaire le permet et que c'est une fonctionnalité utile, nous avons voulu avoir la possibilité de définir et déréférencer des pointeurs de pointeurs. Cela nous a posé beaucoup de problèmes et ce n'est toujours pas parfait comme le montre le test `testPointeurMultipleEtrange.rat` de la passe de génération de code.

Déjà, dans notre implémentation, l'opération *new* pourrait être mieux adaptée aux pointeurs récursifs. En effet l'allocation de la mémoire d'un pointeur de pointeur s'arrête au premier pointeur. C'est-à-dire qu'on alloue seulement la case mémoire du pointeur mais pas celle du type passé au pointeur de pointeur. Il faudrait faire des `malloc` de manière récursive pour allouer une case mémoire de 1 à tous les "pointeurs intermédiaires" puis une case mémoire de la taille du type final et chaîner toutes ces cases en mettant correctement les adresses des cases successives dans chaque pointeur.

Cependant, on peut considérer que c'est au programmeur de bien initialiser ses pointeurs avec une valeur pour chaque case intermédiaire de pointeurs de pointeurs sous peine de voir son programme faire des choses étranges.

Ensuite, comme mentionné plus haut, il faudrait pouvoir déréférencer proprement des pointeurs récursifs sans le paramètre `isDeref`.

3.3 Tests

3.3.1 Passe gestion des identifiants

- `testAffectationPointeur1.rat` : déclaration nouveau pointeur vide (OK)
- `testAffectationPointeur2.rat` : déréférencement pointeur non déclaré (KO)
- `testAffectationPointeur3.rat` : affectation valeur null à pointeur (OK)
- `testAffectationPointeur4.rat` : déclaration et affectation nouveau pointeur avec référencement (OK)
- `testAffectationPointeur5.rat` : déréférencement pointeur (OK)
- `testAffectationPointeur6.rat` : déclaration et affectation de pointeur de pointeur (OK)
- `testDeclarationFonctionPointeur.rat` : déclaration de fonctions avec pointeur comme paramètre et pointeur comme retour (OK)
- `testUtilisationFonctionPointeur1.rat` : déréférencement pointeur passé en paramètre dans corps d'une fonction (OK)
- `testUtilisationPointeur2.rat` : référencement d'une variable passée en argument d'une fonction (OK)
- `testUtilisationPointeur3.rat` : référencement d'une variable non passée en argument d'une fonction (KO)

- testUtilisationPointeur1.rat : utilisation pointeur null (OK)
- testUtilisationPointeur2.rat : utilisation référencement multiples (OK)
- testUtilisationPointeur3.rat : référencement d'identifiant non déclarée (KO)
- testUtilisationPointeur4.rat : utilisation pointeur déclaré et à valeur initialisé (OK)
- testUtilisationPointeur5.rat : référencement pointeur (OK)
- testUtilisationPointeur6.rat : référencement identifiant invalide (KO)

3.3.2 Passe typage

- testAffectationPointeur1.rat : affectation pointeur entier (OK)
- testAffectationPointeur2.rat : affectation pointeur booléen (OK)
- testAffectationPointeur3.rat : affectation pointeur rat (OK)
- testAffectationPointeur4.rat : affectation pointeur de pointeur entier (OK)
- testAffectationPointeur5.rat : affectation entier booléen par déréréfèrencement (KO)
- testAffectationPointeur6.rat : affectation entier rat par déréréfèrencement (KO)
- testAffectationPointeur7.rat : affectation rat booléen par déréréfèrencement (KO)
- testAffectationPointeur8.rat : affectation rat entier par déréréfèrencement (KO)
- testAffectationPointeur9.rat : affectation booleen entier par déréréfèrencement (KO)
- testAffectationPointeur10.rat : affectation booleen rat par déréréfèrencement (KO)
- testAffectationPointeur11.rat : affectation entier à pointeur d'entier (KO)
- testAffectationPointeur12.rat : affectation pointeur de pointeur d'entier à entier (KO)
- testAffectationPointeur13.rat : affectation null à pointeur rat (OK)
- testAffectationPointeur14.rat : affectation null à pointeur int (OK)
- testAffectationPointeur15.rat : affectation null à pointeur bool (OK)
- testAffectationPointeur16.rat : affectation null à pointeur de pointeur bool (OK)
- testAffectationPointeur17.rat : affectation null/booléen (KO)
- testAffectationPointeur18.rat : affectation null/int (KO)
- testAffectationPointeur19.rat : affectation null/rat (KO)
- testDeclarationPointeur1.rat : declaration pointeur int (OK)
- testDeclarationPointeur2.rat : declaration pointeur rat (OK)
- testDeclarationPointeur3.rat : declaration pointeur bool (OK)
- testDeclarationPointeur4.rat : declaration pointeur de pointeur (OK)
- testDeclarationPointeur5.rat : declaration pointeur rat/new int (KO)
- testDeclarationPointeur6.rat : declaration pointeur int/new rat (KO)
- testDeclarationPointeur7.rat : declaration pointeur int/new bool (KO)
- testDeclarationPointeur8.rat : declaration pointeur bool/new int (KO)
- testDeclarationPointeur9.rat : declaration pointeur bool/new rat (KO)
- testDeclarationPointeur10.rat : declaration pointeur rat/new bool (KO)

- testDeclarationPointeur11.rat : declaration pointeur null sur int (OK)
- testDeclarationPointeur12.rat : declaration pointeur null sur rat (OK)
- testDeclarationPointeur13.rat : declaration pointeur null sur bool (OK)
- testDereferenceNonPointeur1.rat : déréréfencement d'un non pointeur (KO)
- testDereferenceNonPointeur2.rat : double déréréfencement (KO)
- testReferencePointeur1.rat : référence variable (OK)
- testReferencePointeur2.rat : référence pointeur (OK)
- testAffichagePointeur.rat : affichage pointeur (OK)
- testConditionnellePointeur1.rat : condition pointeur sur type (KO)
- testConditionnellePointeur2.rat : condition sur null (KO)
- testOperationPointeur1.rat : opération binaire entre pointeurs (KO)
- testOperationPointeur2.rat : opération unaire sur pointeur (KO)
- testOperationPointeur3.rat : opération binaire entre pointeur et autre type (KO)
- testRetourPointeur1.rat : retour de fonction pointeur (OK)
- testRetourPointeur2.rat : retour de fonction pointeur invalide (KO)
- testParametrePointeur1.rat : paramètres de fonction pointeur (OK)
- testParametrePointeur2.rat : fonction appelée avec mauvais paramètre type pointeur (KO)

3.3.3 Passe placement mémoire

Cette passe n'a pas été modifiée pour les pointeurs, cependant on peut noter que les pointeurs (peut importe le type sur lequel ils pointent) prennent toujours une place de 1 puisque ce sont des entiers contenant une adresse.

- testPointeur1.rat : pointeur sur rationnel (OK)
- testPointeur2.rat : pointeur sur plusieurs types à la suite (OK)
- testPointeur3.rat : exemple plus complexe avec conditionnelle (OK)
- testPointeur4.rat : exemple avec les paramètres d'une fonction (OK)

3.3.4 Passe génération de code

- testModifVar.rat : modification variable à l'aide d'un pointeur (OK)
- testEgalitePointeurNull.rat : utilisation pointeur null (OK)
- testNewPointeur.rat : allocation mémoire d'une variable pointée (OK)
- testPointeurMultiple.rat : utilisation de pointeurs de pointeurs (OK)
- testPointeurMultipleEtrange.rat : exemple étrange d'utilisation de pointeurs de pointeurs, comportement non voulu et à améliorer (OK)

4 Variables globales

4.1 Description

Les variables globales ont été entièrement implémentées et accompagnées de quelques tests mais nous n'avons plus le temps de décrire leur fonctionnement dans ce rapport...

4.2 Implémentation

4.3 Tests

4.3.1 Passe gestion des identifiants

4.3.2 Passe typage

4.3.3 Passe placement mémoire

4.3.4 Passe génération de code

5 Variables statiques locales

6 Paramètres par défaut

7 Conclusion

7.1 Difficultés rencontrées

La compréhension du sujet et du compilateur demandé a été une première difficulté importante. En effet, malgré des ajouts demandés par le sujet, il a fallu être imaginatif et comprendre parfaitement le cas d'utilisation du langage RAT et du compilateur que l'on veut produire car on peut toujours aller plus loin et ajouter des fonctionnalités ou voir un cas d'utilisation qui n'était pas prévu par le sujet mais évident dans d'autres langages de programmation.

Ensuite, la connaissance du langage assembleur TAM a aussi posé problème. C'est un langage qu'on ne connaît pas et qu'on n'a pas étudié dans son intégralité, en particulier les primitives utilisables en TAM et leurs comportements exactes. Ceci rejoint aussi la difficulté à comprendre la gestion des registres et de la mémoire qui nous a posé problème à certains moments. Tout cela prouve le besoin d'avoir une expertise profonde des systèmes et de l'architecture des ordinateurs pour le développement d'un compilateur.

De plus, nous avons éprouvé des difficultés à trouver des tests corrects, suffisamment exhaustifs sans en faire trop et efficaces pour tester des fonctionnalités précises. Cela relève plus de la complexité du génie logiciel.

Et enfin, nous avons eu un problème d'organisation pour pouvoir mener ce projet à terme en partie à cause du fait que Victor est à l'étranger et qu'il était compliqué de s'organiser correctement.

7.2 Pour aller plus loin

Comme expliqué dans les sections sur chaque ajout, plusieurs fonctionnalités ou améliorations auraient pu être possible.

En particulier, la gestion de la mémoire n'est pas très développée : on pourrait avoir des opérations d'allocation dynamique comme `malloc`, `calloc`, `realloc` comme en C ainsi que un moyen pour libérer de la mémoire avec une opération `free`.

Un autre aspect des pointeurs que l'on aurait pu ajouter est le test d'égalité entre `null` est un booléen, `null` serait considéré comme un booléen valant faux dans une condition.

De plus, ici les fonctions retournent toujours un type alors qu'on a souvent besoin de procédures qui ne renvoient rien (surtout lorsque l'on a les pointeurs pour modifier des variables). On pourrait alors ajouter le mot *void* pour préciser que la fonction ne renvoie rien.